ALGORITMOS E PROGRAMAÇÃO



Neiva Kuyven

ORDENAÇÃO DE DADOS

INTRODUÇÃO

Olá!

Neste texto você vai estudar os principais algoritmos de ordenação de dados existentes na literatura. Primeiramente, serão abordados esses algoritmos de forma conceitual, a fim de prover o entendimento da importância e a utilidade dos algoritmos de ordenação de dados. Posteriormente será apresentada uma análise do funcionamento peculiar de cada um dos algoritmos, ilustrando o seu funcionamento através de exemplos didáticos. Por fim, ainda neste capítulo, serão apresentadas estratégias para identificação do algoritmo de ordenação de dados mais adequado a cada contexto.

OBJETIVOS DE APRENDIZAGEM

Ao final desta unidade você deve apresentar os seguintes aprendizados:

- Diferenciar alguns dos métodos de ordenação interna de dados.
- Reconhecer o funcionamento de alguns dos métodos de ordenação interna de dados.
- Identificar os benefícios da aplicação da ordenação de dados na solução de problemas do dia a dia.

PRINCIPAIS ALGORITMOS DE ORDENAÇÃO DE DADOS

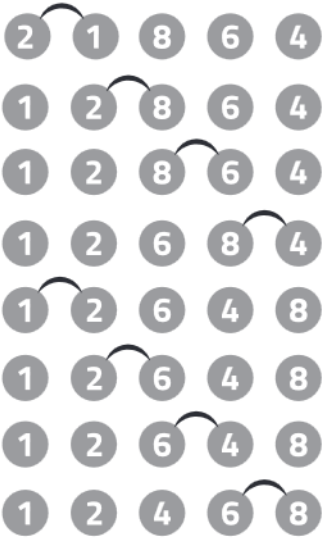
Ao visualizarmos a pluralidade na sentença algoritmos de ordenação de dados, podemos facilmente nos questionar sobre o motivo da existência de mais de um algoritmo destinado ao mesmo fim. Isto porque cada algoritmo possui suas características peculiares, contudo, todos têm um único e principal objetivo, que é ordenar de maneira correta os dados. Nesta seção será analisado cada um dos métodos de ordenação para que, ao final deste estudo, você esteja apto a diferenciar os métodos de ordenação de dados.

ORDENAÇÃO POR BOLHA (*BUBBLE SORT*)

Dentre todos, este é o algoritmo com o funcionamento mais simples de entender e de implementar. A sua estrutura básica consiste na comparação de um elemento com o elemento do índice imediatamente posterior e na verificação dos valores armazenados nos índices. Como exemplo, o objetivo

deste algoritmo pode ser ordenar elementos de forma crescente, da esquerda para a direita. Na comparação, caso o valor da esquerda seja maior, o algoritmo de bolha efetua a troca dos elementos: (vetor[x] < vetor[x+1]).

Este algoritmo é indicado apenas para situações em que o arquivo que se deseja ordenar for pequeno, pois, devido a este algoritmo não otimizar as comparações e comparar em todos os casos todos os elementos do vetor, ele tem um desempenho inferior aos demais algoritmos. Veja, na imagem a seguir, que são necessárias doze iterações para ordenar um algoritmo de 5 elementos, sendo que, caso o menor número estivesse no último elemento, seriam necessárias pelo menos 25 iterações, uma vez que a complexidade do algoritmo é exponencial.



Nova verificação ao final, para conferir se todos estão realmente ordenados...

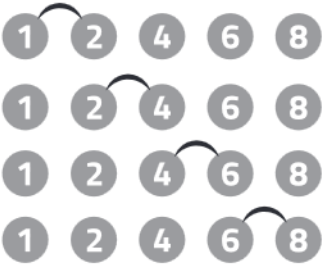


Figura 1 - Execução do algoritmo de bolha

ORDENAÇÃO POR SELEÇÃO (S*ELECTION SORT*)

Neste exemplo, o algoritmo tem por objetivo colocar o menor número na primeira posição do vetor. Diferente do *bubble sort*, que percorre todo o vetor comparando pares de elementos, o *selection sort* seleciona um elemento e o compara com todos os valores do vetor, verificando se ele é o menor. Contudo, a ordenação por seleção, assim como a ordenação por bolha, não possui um bom desempenho para grandes trabalhos de ordenação. Veja, na ilustração a seguir, as etapas necessárias para ordenar o vetor utilizando o algoritmo de ordenação por seleção.

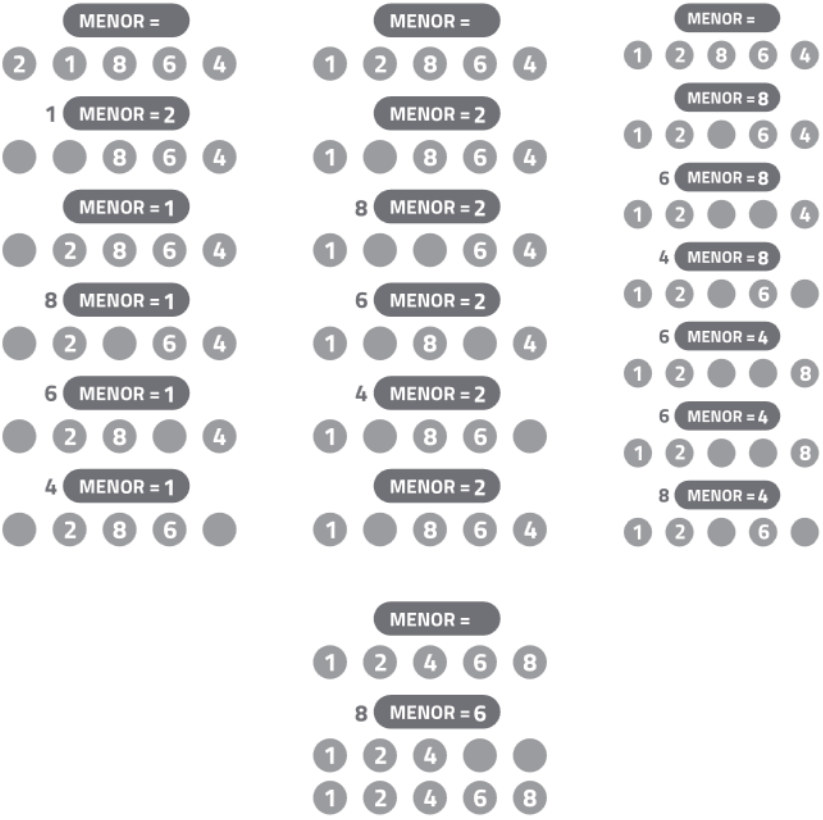
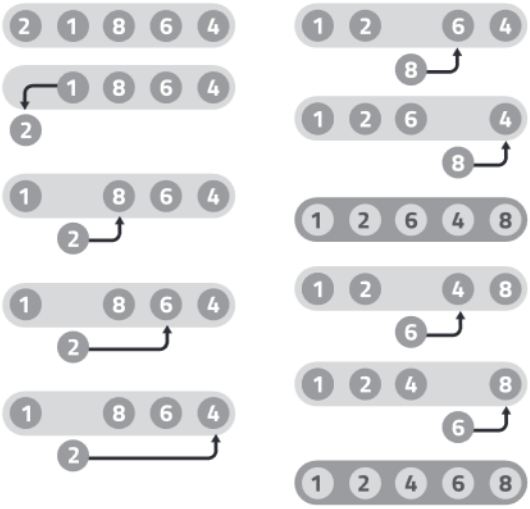


Figura 2 - Execução do algoritmo por seleção

ORDENAÇÃO RÁPIDA (*QUICKSORT*)

O algoritmo de ordenação *Quicksort*, como já diz o próprio nome, é o que possui a melhor eficiência e, por isso, maior velocidade na ordenação de grandes arquivos.



OUTROS ALGORITMOS DE ORDENAÇÃO DE DADOS

Além dos algoritmos já vistos (ordenação por bolha, ordenação por seleção e ordenação rápida), existem outros algoritmos de ordenação de dados, como o de ordenação por inserção (*Insertion Sort*) e o de ordenação por intercalação (*Merge Sort*).

***Insertion Sort*:** Consiste em inserir o elemento já em sua posição correta; dessa forma, sempre que o algoritmo verificar que o elemento do índice imediatamente posterior (x+1) é inferior ao elemento do índice imediatamente anterior (x), além de efetuar a troca entre estes dois elementos, também irá verificar se algum dos elementos anteriores é maior e, então, inserir o valor no local adequado. Por exemplo:

[5] [8] [7] [6] [3] - Vetor inicial.

[5] [8] [7] [6] [3] – O algoritmo seleciona o primeiro elemento, no caso, o número 5 e compara com o elemento imediatamente posterior, no caso, o elemento 8. Como 5 < 8, não é efetuada nenhuma troca.

[5] **[8]** [7] [6] [3] – O algoritmo seleciona o segundo elemento, no caso, o número 8 e compara com o elemento imediatamente posterior, no caso, o elemento 7. Como 8 > 7, é efetuada a troca entre eles. Contudo, antes de inserir o 7 em sua nova posição, o programa vai verificar se o elemento à esquerda do 8 é inferior ao 7; como é inferior, apenas insere o 7.

[5] [7] **[8]** [6] [3] – O algoritmo seleciona o terceiro, no caso, o número 8 e compara com o elemento imediatamente posterior, no caso, o elemento 6. Como 8 > 6, é efetuada a troca entre eles. Contudo, analisando o elemento imediatamente anterior ao 8, que é o 7, verifica-se que 7 é maior que 6 e, por isso, é feita nova troca.

[5] [6] [7] [8] [3] – O algoritmo testa o quarto elemento, no caso, o 8. Como o elemento imediatamente posterior contém o valor 3, é efetuada a troca. Contudo, o valor 3 também é inferior aos elementos anteriores ao 8 (7, 6, 3 e 5) e, por isso, são efetuadas novas trocas até que o valor 3 chegue no seu lugar. Após três novas trocas, o vetor ficará assim: [3] [5] [6] [7] [8].

Merge Sort: Este algoritmo usa a estratégia de dividir para conquistar, ou seja, divide o vetor em partes e ordena dentro de cada parte. Veja o exemplo a seguir:

[5] [3] [2] [4] [7] [1] [0] [6] - Formação inicial do vetor.

[5][3][2][4] [7][1][0][6] - O vetor é dividido em dois grupos.

[5][3] [2][4] [7] [1] [0] [6] - O vetor é novamente dividido, agora em quatro grupos, atingindo, assim, a máxima divisão possível.

[3][5] [2][4] [1][7] [0][6] – O algoritmo realiza a ordenação de cada grupo.
[3][5][2][4] [1][7][0][6] - O algoritmo realiza um reagrupamento.

[2][3][4][5] [0][1][6][7] - O algoritmo realiza a ordenação nos subgrupos maiores através da comparação dos primeiros elementos de cada grupo menor.

Veja a sequência de passos para ordenação entre os grupos:

[2] [3][4][5] **[0]** [1][6][7] - Comparação do primeiro elemento do primeiro grupo com cada elemento do segundo grupo. Se algum dos elementos do segundo grupo for menor que o primeiro elemento do primeiro grupo, é feita a troca de posição. Ao final de todas as trocas tem-se o vetor ordenado.

[0] [1] [2] [3] [4] [5] [6] [7]

SAIBA+

SANTOS, Tiago. **Bubble Sort**. Disponível em: <https://www.youtube.com/watch?v=lIX2SpDkQDc>. Acesso em: 02 jan. 2017.

SANTOS, Tiago. **Insertion Sort**. Disponível em: <https://www.youtube.com/watch?v=-Z00it6Nkz8>. Acesso em: 02 jan. 2017.

SANTOS, Thiago. **Merge Sort**. Disponível em: <https://www.youtube.com/watch?v=cDNqk4tdvqQ>. Acesso em: 02 jan. 2017.

SANTOS, Tiago. **Selection Sort**. Disponível em: <https://www.youtube.com/watch?v=BSXIolKg5F8>. Acesso em: 02 jan. 2017.

REFERÊNCIAS

HONORATO, Bruno de Almeida. **Algoritmos de Ordenação – Análise e Comparação.** Disponível em: <http://www.devmedia.com.br/algoritmos-de-ordenacao-analise-e-comparacao/28261>. Acesso em: 02 jan. 2017.

Conteúdo:

